Masterless Muon: Training 4-bit Weights Without a Full-Precision Copy

Malcolm Frei
Independent researcher
malcolmf830@gmail.com

October 2026

Abstract

Low-bit training normally keeps a full-precision master copy of every weight, because a single optimizer step is far smaller than the spacing of a 4-bit grid. ECO removes the copy by feeding each step's rounding error back into the optimizer's momentum. We show that this does not work for Muon. Muon orthogonalises its momentum, so its update has a nearly fixed size and an error added to the momentum is reshaped on the way to the weights. On GPT-2 small, at the injection size ECO prescribes, about 5% of the error arrives and the update loses the gradient's direction. We propose the *ticker*, which keeps the error beside each weight instead, as an 8-bit carry measured in units of that weight's own grid spacing. The weights are 4-bit values throughout training, and the training state costs 2.78 bytes per weight. In continued training of GPT-2 small, the ticker ends 0.0013 (mean of 3 seeds) above training the same 4-bit model with a float copy and 0.08 below ECO run with AdamW, at a third of ECO's memory. Training in bf16 and rounding afterwards ends 0.04 worse with round-to-nearest and 0.01 worse with GPTQ. Trained from scratch at 45M parameters, the ticker stays within 0.013 of the float copy, but there bf16 training followed by GPTQ matches it. In full-parameter reinforcement learning of Qwen3.5-0.8B on GSM8K, every weight learns while stored in 4 bits and test accuracy rises by 20 points, yet the final model is 3.7 points behind bf16 training and misses the threshold we set in advance, mostly because rounding the starting model to nearest costs 6.5 points (3.5 with GPTQ). Every experiment and threshold was registered before it ran.

1 Introduction

Hardware and serving systems increasingly use 4-bit weights [21, 24], but training a model whose weights are 4-bit numbers almost always means training a different, full-precision model. The standard recipe keeps an fp32 or bf16 *master copy* of each weight. The optimizer adds its update to the copy, and the 4-bit weights used in the forward pass are re-rounded from it at every step. The copy is needed because a typical update is far smaller than the distance between neighbouring 4-bit values, so rounding it on its own would erase it. The price is memory and a subtler mismatch, because the model being trained is not the model being served. In reinforcement learning (RL) the mismatch becomes visible, because a quantized generator writes the answers that a full-precision trainer then scores, and the difference has to be corrected for [19, 29].

ECO [20] removes the master copy for SGD with momentum and for Adam. After each step, the rounding error is added into the momentum with a scale chosen so that the optimizer re-applies it over the following steps. This works because both optimizers are linear in the momentum (Adam once its second moment is fixed), so an error placed in the momentum reappears unchanged in the

weights. Muon [13, 18], now a common choice for the matrices of language models, is not linear in its momentum. It replaces the momentum by an approximately orthogonal matrix with the same singular vectors, which gives its update a nearly fixed size whatever the size of the momentum. Section 3 derives what this does to an injected error and measures it on GPT-2 small. The error loses part of its energy, is redistributed across singular directions, and at the sizes ECO uses it overwrites the gradient.

We therefore keep the error beside each weight. Each weight stores a 4-bit value and a small integer *carry*, the part of the weight's intended position that the grid cannot yet show, counted in units of the gap to the neighbouring grid value. Each step adds the update to the weight and its carry, rounds the sum to the nearest grid value, and stores what is left as the new carry. With an exact carry this is Kahan's compensated summation [14] applied to 4-bit weights, and it reproduces training with a float copy exactly. The practical question is how few bits the carry can have. The same idea, a few-bit residual buffer per weight, was proposed days before this work for binary networks under the name Error Diffusion [1]; the ticker is its multi-level form for 4-bit grids with block scales.

With an 8-bit carry and 8-bit momentum, the ticker trains GPT-2 small to within about a thousandth of a nat of a float copy, well ahead of ECO and of naive stochastic rounding, and a 45M-parameter model from scratch to within about a hundredth. Against the strongest alternative we found, training in bf16 with stochastic rounding and rounding to 4 bits at the end, the result depends on the setting. The ticker is ahead in continued training even when the final rounding uses GPTQ [6], while from scratch GPTQ closes the gap. In RL on a 0.8B model the ticker trains every weight in 4 bits and improves the 4-bit model by twenty points, but does not fully recover what rounding the starting model cost.

Contributions.

1. An analysis of why momentum-based error feedback fails when the update is orthogonalised (Proposition 1), and a measurement on GPT-2 small that needs no training (Section 3).
2. The ticker: 4-bit weights trained without a master copy, at 2.6–2.8 bytes per weight of training state, evaluated against float copies, ECO with Muon and with AdamW, stochastic rounding, and post-training rounding with GPTQ (Sections 2 and 5.1–5.2).
3. A pre-registered test of a noise model for the carry width. The model fails both tests; the logged flip rates point to a different mechanism, which we describe (Sections 4 and 5.3).
4. Full-parameter RL with every decoder weight stored in 4 bits, compared with full-precision training and with a frozen 4-bit model plus LoRA, and a measurement of training memory with packed storage (Sections 5.5–5.6).

Every experiment, its seeds and its success criterion were written into a dated plan before the experiment ran. Table 3 lists the criteria and their outcomes, and Appendix C lists every change made to the plan and when. The code, the plan and the log of every run are public (`https://github.com/Nibbles0n/masterless`).

2 Method

2.1 The 4-bit format

Each weight matrix $W \in \mathbb{R}^{m \times n}$ (output × input) is split into blocks of 16 consecutive weights along the input dimension. A block has one scale $s = \max_i |w_i|/6$, and each weight is $w = s\,x$ with x

from the signed E2M1 value set

$$x \in \{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\},$$

as in NVFP4 [21]; MXFP4 [24] uses the same values with a power-of-two scale per 32 weights. RTN$(\cdot)$ rounds to the nearest grid value. For a grid value x we write $g_+(x)$ and $g_-(x)$ for the gaps to the next value up and down, which are 0.5, 1 or 2 times s (beyond ± 6 we use 2). We keep the scale in fp32 during training; NVFP4 stores it in FP8 (E4M3) with one fp32 scale per tensor, which saves 0.19 bytes per weight. Only the matrices inside the transformer blocks are stored this way and trained by Muon. Embeddings, norms and biases stay in floating point and are trained by AdamW, as is usual for Muon.

2.2 The ticker

Let $\hat{w}$ be a weight's stored 4-bit value, c its carry, η the learning rate and Δ the optimizer's update for this step. One step is

$$\tilde{w} = \hat{w} + c - \eta\Delta \qquad \text{(target position)} \tag{1}$$

$$\hat{w}' = \mathrm{RTN}_{s'}(\tilde{w}), \quad s' = \max_{i \in \text{block}} |\tilde{w}_i|/6 \qquad \text{(nearest value on the new grid)} \tag{2}$$

$$r = \tilde{w} - \hat{w}' \qquad \text{(remainder)} \tag{3}$$

$$k' = \mathrm{SR}\left(2L \, \frac{r}{g_\pm(\hat{w}')}\right) \in \{-L, \ldots, L\} \qquad \text{(new carry, an integer)} \tag{4}$$

$$c' = \frac{k'}{2L} \, g_\pm(\hat{w}') \qquad \text{(the value it stands for)} \tag{5}$$

where $g_\pm$ is the gap on the side of r's sign, $L = 2^{b-1} - 1$ for a b-bit carry, and SR rounds to one of the two neighbouring integers at random, with probabilities that make it unbiased. The old carry c is decoded with the old scale and gaps. The new scale s' is computed from the 16 targets of the block before rounding, and every weight of the block is rounded to the grid of s' and has its remainder measured against that grid, so a change of scale is absorbed into the 16 new carries rather than rescaling them separately. The block's largest target equals $6s'$ exactly, so the fp32 scale holds it without error and its carry is zero.

Because $\hat{w}'$ is the nearest grid value, $|r| \le g_\pm/2$, so $|k'| \le L$ and the carry never overflows. A weight moves to the next grid value ("ticks") when its carry passes half a gap. Since the carry is measured relative to the weight's own gap, small weights (fine gaps) and large weights (coarse gaps) get the same $2L$ levels of resolution; at $b = 8$ the resolution is $1/254$ of a gap.

With c kept exactly in fp32, $\hat{w} + c$ equals the float-copy weight at every step for the same sequence of updates, which is Kahan summation [14] with a 4-bit main term. The ticker quantizes the compensation term in units that follow the main term's local resolution, so its only departure from a float copy is the random rounding of the carry, and of the momentum (below).

2.3 8-bit momentum

Muon's momentum is stored as 8-bit integers $(-127 \ldots 127)$ with one fp32 scale per 128 weights of a row, rounded at random after each update. Newton–Schulz runs on the dequantized Nesterov momentum.

Table 1: Training state per weight of the Muon-trained matrices. Scales: one fp32 scale per 16 weights (0.25 bytes per weight) or, in brackets, NVFP4-style FP8 scales (0.0625). Momentum scale: one fp32 scale per 128 weights (0.03).

method	parts	bytes/weight
4-bit + float copy, AdamW	fp32 copy + two fp32 moments	12
ECO-AdamW	4-bit + scale + two fp32 moments	8.75
4-bit + float copy, Muon	fp32 copy + fp32 momentum	8
Kahan (exact carry)	4-bit + scale + fp32 carry + 8-bit mom.	5.78
bf16-SR	bf16 weight + 8-bit momentum	3.03
ticker, 8-bit carry	4-bit + scale + 8-bit carry + 8-bit mom.	**2.78** (2.59)
naive / ECO-Muon	4-bit + scale + 8-bit mom.	1.78 (1.59)

2.4 Memory

Table 1 counts the training state of the block matrices: weights plus optimizer state. Gradients and activations are the same for all methods and are not counted.

3 Why error feedback fails with Muon

3.1 Muon's update has a nearly fixed size

For a matrix W with gradient G_t, Muon [13] keeps a momentum $B_t = \beta B_{t-1} + (1-\beta)G_t$ and orthogonalises the Nesterov combination $U_t = (1-\beta)G_t + \beta B_t$:

$$W \leftarrow W - \eta\,\alpha\,\mathrm{NS}(U_t), \qquad \alpha = \sqrt{\max(1, m/n)},$$

where NS is five steps of the quintic Newton–Schulz iteration $X \leftarrow aX + (bA + cA^2)X$ with $A = XX^\top$, $(a, b, c) = (3.4445, -4.7750, 2.0315)$, started from $X = U/\|U\|_F$. If $U = P\Sigma Q^\top$ is a singular value decomposition, then $\mathrm{NS}(U) = Pf(\Sigma/\|U\|_F)Q^\top$ for a fixed odd polynomial f that maps every singular value that is not tiny into roughly $[0.7, 1.2]$. NS thus approximates the polar factor $\mathrm{polar}(U) = PQ^\top$.

Two properties follow. First, $\mathrm{NS}(cU) = \mathrm{NS}(U)$ for every $c > 0$. Second, $\|\mathrm{NS}(U)\|_F \approx \bar{f}\sqrt{\min(m, n)}$, where $\bar{f}$ is the root-mean-square of f over the singular values, so each step moves the matrix by $\eta\alpha\bar{f}\sqrt{\min(m,n)}$ in Frobenius norm, or by $\eta\alpha\bar{f}/\sqrt{\max(m,n)}$ per weight (root-mean-square), whatever the size of the gradient. On GPT-2 small we measure $\bar{f} = 0.905$ (standard deviation 0.066, range 0.69–0.99) over 48 matrices and 40 steps, while the momentum's own root-mean-square varies by a factor of 49 (Figure 1c). Multiplying the momentum by 10^3 or 10^{-3} changes the update by less than 0.02%.

3.2 What happens to an injected error

ECO adds $\alpha_{\mathrm{ECO}}\,e$ to the momentum, where e is the rounding error of the last step and $\alpha_{\mathrm{ECO}} = \eta^{-1}(1-\beta^{-1})$ for SGD with momentum; for Adam the same coefficient is multiplied by Adam's per-weight step size. The coefficient is negative because $\beta < 1$, and the next step subtracts η times the momentum, so the error returns with the correct sign. For an optimizer whose update is linear in the momentum, this places $(1-\beta)e$ into the next step and e into the sum of all later steps, which returns the weight to where a float copy would have put it. For Muon the response is governed by the derivative of the polar factor.

Proposition 1 (First-order response of the polar factor)**.** *Let $U \in \mathbb{R}^{m\times n}$ with $m \le n$ have full rank and thin singular value decomposition $U = P\Sigma Q^\top$, where P is $m \times m$ orthogonal, $Q^\top Q = I_m$ and $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_m)$ with $\sigma_i > 0$. For a perturbation δ let $\tilde\delta = P^\top \delta Q$. Then*

$$\mathrm{polar}(U+\delta) = PQ^\top + PKQ^\top + P\Sigma^{-1}P^\top\delta\,(I - QQ^\top) + O(\|\delta\|^2), \qquad K_{ij} = \frac{\tilde\delta_{ij} - \tilde\delta_{ji}}{\sigma_i + \sigma_j}. \tag{6}$$

In particular, the symmetric part of $\tilde\delta$ has no first-order effect, and the antisymmetric part is divided by $\sigma_i + \sigma_j$.

The proof is in Appendix A; the formula is a form of the standard perturbation result for the polar decomposition [10, Ch. 8]. Compared with passing δ through at the update's average gain $\kappa = \|\mathrm{NS}(U)\|_F / \|U\|_F$, as a linear optimizer would, it implies three distortions.

1. **Part of the error is erased.** The symmetric part of $\tilde\delta$ would only change the singular values, which orthogonalisation sets to about 1. For the actual 4-bit rounding error of GPT-2 small this part holds 22% of the energy (7–50% by matrix).

2. **The rest is reweighted.** Relative to κ, each component is scaled by about $\sigma_{\mathrm{rms}}/\sigma$ for the singular values involved, so directions with small singular values are amplified and the few large ones, which carry most of the gradient signal, are damped. Momentum spectra are heavy-tailed, so the delivered correction points away from the error. Newton–Schulz bounds the amplification: the slope of f at zero is $a^5 \approx 485$ on the normalised input, at most about $485/(\bar f\sqrt{\min(m,n)}) \approx 19$ times κ for the 768-wide matrices of GPT-2 small.

3. **Large errors saturate.** Because NS is scale-invariant, $\mathrm{NS}(U+\delta) \to \mathrm{NS}(\delta)$ as $\|\delta\|/\|U\| \to \infty$, so the change in the update is bounded by about $2\bar f\sqrt{\min(m,n)}$, and the share of the intended correction that arrives falls like $\|U\|/\|\delta\|$.

No single injection scale avoids all three. A small injection is bent and amplified, a large one is bent and truncated, and in between it competes with the gradient for the direction of the update.

3.3 Measurement on GPT-2 small

We take pretrained GPT-2 small [23], compute real gradients of its 48 block matrices on FineWeb [22] windows, and build Muon's Nesterov momentum over 40 steps (2 windows of 1,024 tokens each). We then add to U an error whose direction is each matrix's actual 4-bit rounding remainder $W - \mathrm{RTN}(W)$, at sizes from 0.01 to 10 times $\|U\|$, and compare the change in the update with the error (Figure 1, Table 2). For SGD with momentum the error passes through exactly. Adam with ECO's per-weight rule also passes it through exactly, because ECO multiplies the error by the same per-weight factor that Adam divides by; injected without that factor, Adam rescales it weight by weight (cosine 0.80).

For Muon, the cosine between the delivered and the intended correction is 0.65–0.77 at every size (0.48–0.91 for single matrices). The share of the intended correction that arrives along the error falls from 7.6 at size 0.01, where small errors are amplified, through 0.90 at size 1 to 0.093 at size 10. At the size ECO's rule gives for this error at learning rate 10^{-3}, 22 times the momentum on average, 4.6% of the intended correction arrives, and the update keeps a cosine of only 0.015 with the error-free update. Even an error one tenth the size of the momentum turns the update by about 40° (cosine 0.76 with the error-free update). A random error direction behaves the same way, so the effect comes from Newton–Schulz itself. An earlier training-time measurement on a small character-level model found the same cosine, about 0.69, and ECO with Muon there ended worse than no error feedback at all (Appendix B).

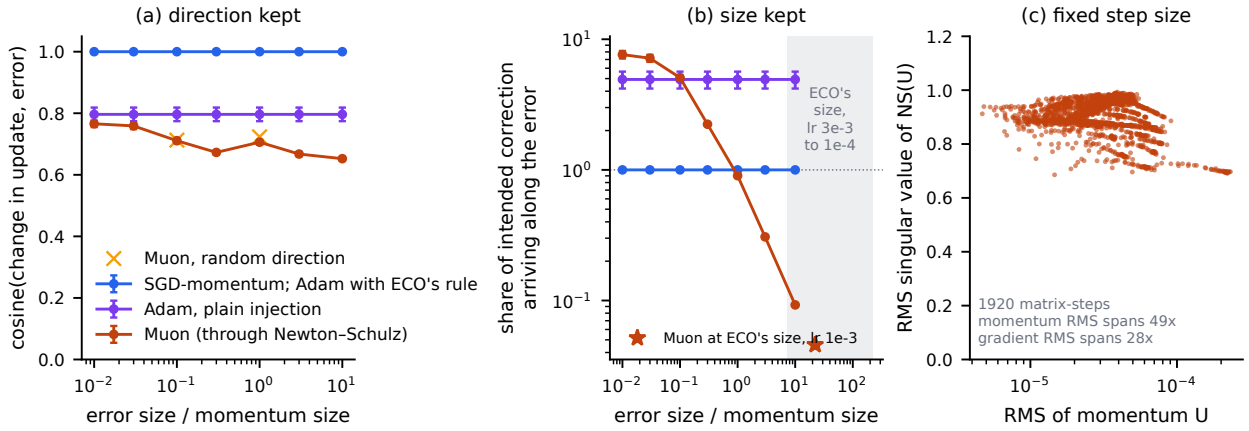


Figure 1: Error added to the momentum, pretrained GPT-2 small, 48 block matrices (mean ± standard error over matrices). (a) Cosine between the change in the update and the injected error. (b) Share of the intended correction that arrives along the error (= cosine × size ratio; 1 means it passes through intact); the grey band is the size ECO's rule gives at learning rates $3 \cdot 10^{-3}$ to 10^{-4}. (c) Muon's step size: the root-mean-square singular value of NS(U) against the root-mean-square of the momentum U, for every matrix and step.

4 A noise model for the carry width

Before the main experiments we registered a simple model for how many carry bits are needed. The ticker differs from a float copy only through the random rounding of the carry (and of the momentum, which the model ignores). Each step rounds the carry to a multiple of $u_b = g/(2L) \approx g/2^b$, where g is the weight's gap. Stochastic rounding is unbiased, and its error on one step has variance $p(1-p)u_b^2 \le u_b^2/4$. Fresh random bits are drawn at every step, so the errors are independent, and after T steps they add up to a drift with standard deviation at most

$$D_b(T) = \frac{u_b}{2}\sqrt{T} \approx \frac{g}{2^b} \cdot \frac{\sqrt{T}}{2}.$$

Muon moves each weight by about $\eta\,\alpha\bar{f}/\sqrt{\max(m,n)} =: \eta\,\bar{u}$ per step (Section 3). If the weight keeps moving in one direction, its intended travel is $\eta\bar{u}T$, and the ratio

$$\rho = \frac{D_b(T)}{\eta\,\bar{u}\,T} = \frac{g}{2^{b+1}\,\eta\,\bar{u}\,\sqrt{T}}$$

measures how much of that travel is noise. The model says a carry of b bits is enough when ρ is below a threshold ρ^*, which we fit on two learning rates and test on two others (Section 5.3).

The model rests on five assumptions. (i) Rounding errors at different steps are independent; this holds for the random bits, but the weight's position also feeds back into the gradient. (ii) Travel is linear in T; if the update's sign flips from step to step, travel grows more slowly and ρ underestimates the damage. (iii) The gap g is fixed, whereas in practice it changes when a weight moves to another grid cell or the block's scale changes. (iv) The momentum's 8-bit rounding noise is ignored. (v) Damage to the loss grows with ρ; the model itself predicts only a per-weight noise level. The model suits Muon better than Adam, because the fixed step size makes $\bar{u}$ a constant of the matrix shape, whereas Adam's per-weight step $|\hat{m}|/\sqrt{\hat{v}}$ varies with each weight's gradient signal-to-noise ratio.

Table 2: Survival of an error added to the momentum, with the actual 4-bit rounding remainder as direction. Mean over the 48 block matrices of GPT-2 small, momentum built over 40 steps. Share arriving: the part of the change along the error, as a share of the error passed through at the optimizer's average gain.

optimizer	cosine	size ratio	share arriving along the error
SGD-momentum	1	1	1
Adam, ECO's rule	1	1	1
Adam, plain injection	0.80	7.3	4.9
Muon, error = 0.1× momentum	0.71	7.1	5.1
Muon, error = 1× momentum	0.71	1.28	0.90
Muon, error = 10× momentum	0.65	0.142	0.093
Muon, ECO's size at $\eta = 10^{-3}$ (22×)	0.65	0.071	0.046

5 Experiments

Protocol. The experiments are labelled P1–P6 as in the registered plan. Within a table, all methods run on the same GPU type and number format. We ran 3 seeds where methods were expected to be close and 1 seed where they were expected to be far apart; a single-seed difference counts only if it exceeds 10 times the largest seed standard deviation in the same table. Language-model losses are validation cross-entropy in nats per token on 256 fixed FineWeb windows of 1,024 tokens. A ± after a loss difference is the paired standard error over those 256 windows, which captures evaluation noise for a fixed model; seed-to-seed variation is reported separately as a standard deviation over seeds. RL intervals are 95% intervals from the paired standard error over the 1,319 test questions, with each question's 0/1 result averaged over seeds. Hardware, run times and total compute, including runs that were discarded, are in Appendix E.

5.1 P1: continued training of GPT-2 small

Setup. Pretrained GPT-2 small [23] is trained on 100M FineWeb tokens (3,050 steps of 32×1024 tokens), with Muon at 10^{-3} on the 48 block matrices and AdamW at 10^{-4} on the remaining parameters, 5% warm-up and cosine decay to 10%, on one A100 in fp16 with loss scaling, TF32 off and deterministic kernels. We compare the ticker (8-bit carry, 8-bit momentum) with: 4-bit weights trained from a float copy, with Muon and with AdamW; Kahan, the ticker with an exact fp32 carry; bf16 weights with stochastic rounding and 8-bit momentum (bf16-SR), scored both in bf16 and rounded to 4 bits after training; float Muon, scored in fp32 and rounded; naive 4-bit training, which rounds the weights at random with no carry; ECO-Muon, ECO's injection into Muon's momentum; and ECO-AdamW, ECO as its paper runs it. After these runs, we also rounded the float Muon and bf16-SR models (seed 1) and the pretrained starting point to 4 bits with GPTQ [6], with predictions registered before the GPTQ runs. GPTQ rounds one column at a time and shifts each column's error onto the columns not yet rounded, using second-order statistics from 128 calibration windows of training data. Our implementation uses the same 4-bit values and per-16 block scales as every 4-bit method, with each block's scale set from the already-corrected weights.

Results. At 2.78 bytes per weight the ticker ends 0.0013 above the 8-byte float copy (mean of 3 seeds), and 0.0012 above an exact fp32 carry, so an 8-bit carry loses almost nothing (Table 4). The cost of 4-bit weights themselves, measured against float Muon, is +0.021 for the ticker and

Table 3: Criteria registered before each experiment and their outcomes. Differences are in validation loss (lower is better) unless stated; RL differences are in GSM8K test accuracy points.

	criterion	result	outcome
P1a	ticker $\leq$ float copy (Muon) +0.03, each seed	$+0.0011, +0.0006, +0.0023$ (± 0.0005)	met
P1b	ticker at least 0.02 below naive rounding and below ECO-Muon, beyond the single-seed rule	0.160 and 0.037 (± 0.002); rule 0.033	met
P1c	ticker below bf16-SR rounded to nearest, each seed	$0.046, 0.049, 0.043$ (± 0.001)	met
P1d	ticker within 0.01 of the exact fp32 carry	$+0.0012 \pm 0.0005$	met
P1e	ticker below ECO-AdamW by more than 2 s.e., each seed	$0.079, 0.080, 0.080$ (± 0.0014)	met
P2a	ticker $\leq$ float copy +0.03, each seed	$+0.0049, +0.0128, +0.0101$ (± 0.0019)	met
P2c	ticker (mean) below bf16-SR rounded to nearest by more than 2 seed s.d.	1.132 (2 s.d. $= 0.013$)	met
GPTQ	every bf16 or float model rounded with GPTQ above the ticker by more than 2 s.e.; otherwise the claim narrows	P1: $+0.0091$, $+0.0122$ (± 0.0007); P2: $+0.0028 \pm 0.0021$	not met
P3a	rank correlation between ρ and the measured gap ≥ 0.8	0.795	not met
P3b	threshold fitted on two learning rates predicts the smallest working carry within one level on the other two	predicts 12 bits at 10^{-4}, where 6 bits worked	not met
P3c	noise model's predicted carry used for RL	8 bits used instead	not tested
P5a	lower end of the 95% interval of ticker $-$ full precision above -4 points	-3.7, $[-5.2, -2.2]$	not met
P5b	ticker gains at least 5 points over its 4-bit start, interval above 0	$+20.4$, $[+18.0, +22.9]$	met
P6a	ticker's peak memory $\leq 0.35\times$ the standard setup's	0.318	met

+0.020 for the float copy. bf16-SR is the best model while it stays in bf16, but rounded to nearest 4-bit values it ends 0.043–0.049 behind the ticker, and float Muon rounded the same way ends 0.042 behind. GPTQ recovers about half of what rounding to nearest loses (52% for both trained models, 56% for the pretrained start), and both GPTQ models remain behind the ticker, by 0.0091 ± 0.0007 (float Muon) and 0.0122 ± 0.0007 (bf16-SR).

ECO-Muon improves on naive rounding but freezes the 4-bit weights: only 0.0004% of them change per step, against 0.14% for the ticker, and the rate falls to zero by the end of training. The injected error dominates the momentum, so each step mostly points back towards undoing the last rounding (Section 3). Its first run, with Newton–Schulz in fp16 like every other method, overflowed by step 10 because the injected error makes the momentum far larger than usual. The run in Table 4 repeats it with Newton–Schulz in fp32, which can only help it.

ECO in its own setting. ECO-Muon is our adaptation, since the ECO paper gives SGD and Adam only. P1e therefore runs its Adam version [20, Algorithm 3]: the same 4-bit grid, stochastic rounding, which the ECO paper finds works best, the rounding error injected into Adam's first moment, both moments in fp32, and the learning rate, betas and weight decay of the AdamW float

Table 4: P1: final validation loss, GPT-2 small after 100M tokens. The paired standard error of a difference between two models is about 0.0005–0.0014 (Table 3). The largest seed standard deviation among the 3-seed methods is 0.0033.

method	bytes/weight	seed 1	seed 2	seed 3
ticker	2.78	3.2884	3.2892	3.2889
4-bit + float copy, Muon	8	3.2873	3.2886	3.2866
4-bit + float copy, AdamW	12	3.2840	3.2859	3.2846
Kahan (fp32 carry)	5.78	3.2872		
bf16-SR, scored in bf16	3.03	3.2690	3.2720	3.2695
bf16-SR, rounded to nearest	3.03	3.3346	3.3387	3.3322
bf16-SR, rounded with GPTQ	3.03	3.3006		
float Muon, scored in fp32	8	3.2670		
float Muon, rounded to nearest	8	3.3303		
float Muon, rounded with GPTQ	8	3.2975		
ECO-AdamW (stochastic rounding)	8.75	3.3679	3.3693	3.3687
ECO-Muon (Newton–Schulz in fp32)	1.78	3.3253		
naive (stochastic rounding, no carry)	1.78	3.4483		
start: pretrained, in fp32		3.4025		
start: rounded to nearest		3.4620		
start: rounded with GPTQ		3.4285		

Table 5: P2: final validation loss, 45M model trained from scratch on 0.9B tokens.

method	bytes/weight	seed 1	seed 2	seed 3	mean ± seed s.d.
ticker	2.78	3.9127	3.9262	3.9196	3.9195 ± 0.0067
4-bit + float copy, Muon	8	3.9078	3.9133	3.9095	3.9102 ± 0.0028
bf16-SR, scored in bf16	3.03	3.8894			
bf16-SR, rounded to nearest	3.03	5.0515			
bf16-SR, rounded with GPTQ	3.03	3.9155			

copy, not tuned for ECO. This comparison was added after an outside review and registered before it ran (Appendix C). ECO-AdamW ends 0.080 behind the ticker in every seed and 0.084 behind the float copy trained with the same AdamW, against 0.0040 for the ticker at 2.78 bytes per weight instead of 12. It drops to 3.359 in the first 300 steps and then stays flat while the float copy keeps improving. Its weights are not frozen: 1.6% of its 4-bit values change per step. We did not test why it stalls. A plausible cause is that each AdamW step is much smaller than a 4-bit gap, so stochastic rounding adds noise of about a gap at every step, while the injection returns only a $(1 - \beta_1)$ share of each step's error to the weights.

5.2 P2: training from scratch

Setup. A GPT-2-style model with 6 layers of width 512 (45.2M parameters) is trained from random weights on 0.9B FineWeb tokens (27,466 steps of 32×1024 tokens, about 20 tokens per parameter). The Muon learning rate, 0.04 with AdamW at $4 \cdot 10^{-3}$, was chosen from three values by short runs of the float copy and used for every method. The ticker and the float copy ran 3 seeds and bf16-SR one; the bf16-SR model was also rounded with GPTQ.

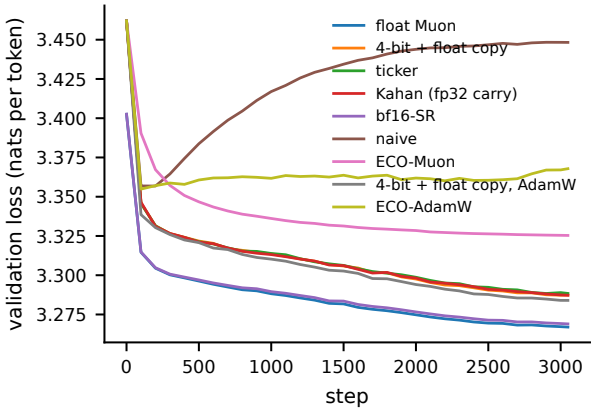


Figure 2: P1: validation loss during training, seed 1. The ticker, Kahan and the 4-bit float copy overlap; naive rounding gets worse after the first steps; ECO-Muon and ECO-AdamW stall.

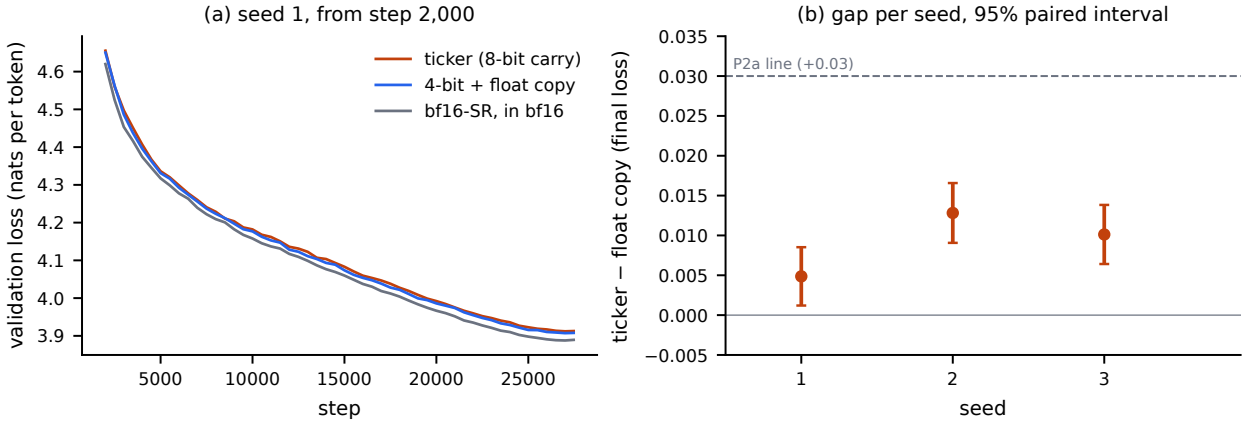


Figure 3: P2: (a) validation loss from step 2,000, seed 1; (b) ticker minus float copy at the end of training, per seed.

Results. From scratch, the gap to the float copy is larger than in continued training (mean +0.009 against +0.0013) and is resolved in every seed, but it stays well inside the registered margin, and the carry never overflowed. The ticker also varies more between seeds (standard deviation 0.0067 against 0.0028), which we attribute to the random rounding of the carry. bf16-SR is again the best model in bf16 (3.889), and rounding it to nearest costs 1.16. GPTQ recovers 98% of that loss, and the GPTQ model scores 3.9155, 0.0028 ± 0.0021 above the ticker's seed 1, which is less than twice its standard error, and inside the range of the ticker's three seeds (3.913–3.926). By the reading registered before the GPTQ runs, the claim that 4-bit training beats training in higher precision and rounding afterwards therefore holds here only against rounding to nearest. At this scale, bf16-SR followed by GPTQ produces an equally good 4-bit model for 3.03 bytes per weight instead of 2.78. We did not test why GPTQ recovers nearly everything here and only about half in continued training. The learning rate is the largest of the three tried, so a larger one might help every method.

Table 6: P3: ticker minus float copy at the same learning rate, 10M tokens, seed 1 (±: paired standard error).

carry bits	10^{-4}	$3 \cdot 10^{-4}$	10^{-3}	$3 \cdot 10^{-3}$
4	$+0.0425 \pm 0.0013$	$+0.0518 \pm 0.0012$	$+0.0513 \pm 0.0012$	$+0.0452 \pm 0.0010$
6	-0.0013 ± 0.0004	$+0.0017 \pm 0.0005$	$+0.0041 \pm 0.0005$	$+0.0022 \pm 0.0006$
8	-0.0005 ± 0.0002	-0.0002 ± 0.0003	-0.0000 ± 0.0003	-0.0006 ± 0.0005
12	-0.0001 ± 0.0001	$+0.0001 \pm 0.0002$	$+0.0004 \pm 0.0003$	$+0.0007 \pm 0.0005$

Table 7: P3: share of 4-bit values that change at each step (%), mean over the logged steps of the second half of training. Computed after the registered analysis, from quantities logged during the runs.

	10^{-4}	$3 \cdot 10^{-4}$	10^{-3}	$3 \cdot 10^{-3}$
4-bit carry	1.57	1.62	1.61	1.63
6-bit carry	0.35	0.36	0.39	0.43
8-bit carry	0.088	0.095	0.111	0.150
12-bit carry	0.008	0.012	0.028	0.076
float copy	0.003	0.010	0.034	0.101

5.3 P3: carry width and learning rate

Setup. GPT-2 small is trained for 10M tokens (305 steps, same schedule shape) at Muon learning rates $\{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}\}$, with carries of $\{4, 6, 8, 12\}$ bits and a float copy at each rate: 20 runs, seed 1. Table 6 gives each run's gap to the float copy, and Figure 4 plots the gap against ρ.

Results. The noise model fails both registered tests. The rank correlation between ρ and the gap is 0.795, just below the required 0.8. The threshold fitted on the two middle rates, $\rho^* = 1.01$, separates working from failing runs there without error, but on the held-out rates it predicts that 12 bits are needed at 10^{-4}, where 6 bits already suffice, two levels off. The damage depends on the carry width and hardly on the learning rate: a 4-bit carry costs about 0.045 at every rate, a 6-bit carry 0.002–0.004, and 8 and 12 bits nothing measurable. Ranked by carry width alone, the runs correlate with the gap at 0.92, a measure chosen after seeing the data. The model predicts the opposite trend, since smaller learning rates need more steps per unit of travel and should accumulate more noise.

The flip rates logged during the same runs suggest why (Table 7). With a float copy the share of 4-bit values that change per step grows thirty-fold from the smallest to the largest learning rate, as expected when weights tick at a speed set by the step size. With a 4-bit carry it is 1.6% at every rate, and at the smaller rates each two extra carry bits divide it by about four. After a weight ticks, its remainder is close to half a gap, so its carry sits next to the rounding boundary. There, the random rounding of the carry moves the target by up to one carry unit (1/14 of a gap at 4 bits), which is much larger than one Muon step at any of these learning rates, so the weight flips back and forth at random instead of following the gradient. The number of weights near the boundary scales with the carry unit, which matches the factor of four per two bits, and the jitter does not depend on the step size, which matches the flat damage at 4 bits. This reading was formed after seeing the data and has not been tested separately; the noise model's small-step correction (variance about $u_b\,|\eta\bar{u}|$ instead of $u_b^2/4$) still predicts a dependence on the learning rate and does

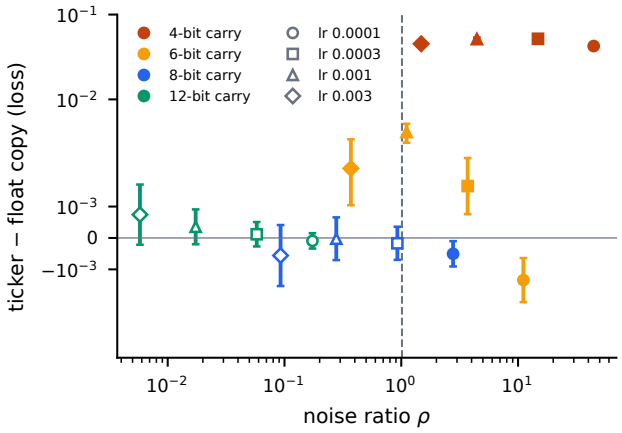


Figure 4: P3: gap to the float copy against the noise ratio ρ (bars: 2 paired standard errors; open markers: within 2 errors of zero). Dashed: the fitted threshold ρ^*.

Table 8: P4: one change at a time (GPT-2 small, 25M tokens, seed 1).

run	final loss	minus ticker	changing per step
ticker	3.3047		0.133%
carry in absolute units (block's largest gap)	3.3059	$+0.0012 \pm 0.0005$	0.101%
weight rounded stochastically instead of to nearest	3.3759	$+0.0713 \pm 0.0012$	31.2%
Error Diffusion's cap (0.25 gap per step)	3.3043	-0.0004 ± 0.0004	0.133%

not explain the flat rows either. An 8-bit carry was sufficient at every learning rate we tried, and the RL experiments use it; the noise model's prediction of a 12-bit carry for RL was recorded but not tested.

5.4 P4: ablations

Starting from the ticker, we changed one design choice at a time (GPT-2 small, 25M tokens, seed 1; these runs had no registered criteria).

Rounding the weight to nearest matters most. With stochastic rounding a third of the weights flip at every step and the loss is 0.071 worse. Measuring the carry in units of each weight's own gap helps a little (0.0012). Error Diffusion's cap on the step never binds at this learning rate, because Muon's steps are far below a quarter of a gap.

5.5 P5: full-parameter RL in 4 bits

Setup. We train Qwen3.5-0.8B (post-trained, non-thinking mode) on GSM8K [2] with GRPO [26]: 8 sampled answers for each of 32 questions per step, reward 1 if the final number is correct, 200 steps, no KL term, one update per step. Answers are generated by vLLM [16] in the same process, with the trained weights copied in as bf16 after every step. Accuracy is measured on the 1,319 test questions with greedy decoding, at the start and at the end. We compare three setups. (a) Full-parameter training with bf16 weights and an fp32 copy, Muon on the decoder matrices. Generator and trainer both use 16-bit weights, so this setup has no quantization mismatch and serves as the full-precision reference. (b) The 4-bit model frozen, with LoRA [11] of rank 32 on every decoder matrix, trained with AdamW at 10^{-5}, as in QeRL [12]. (c) The ticker: every decoder

Table 9: P5: GSM8K test accuracy (%), Qwen3.5-0.8B, 200 RL steps. Gains are 95% intervals from each question's result averaged over seeds and paired across setups.

setup	weights as scored	start	end (per seed)	gain
(a) bf16 + fp32 copy	bf16	54.7	72.9, 71.1, 72.9	$+17.6 \pm 2.4$
(b) 4-bit frozen + LoRA	4-bit + bf16 adapter	48.1	70.6, 70.2, 72.8	$+23.1 \pm 2.5$
(b) merged, rounded to 4 bits	4-bit	48.1	–, 47.6, 47.8	-0.4 ± 2.0
(c) ticker	**4-bit**	48.1	69.7, 67.7, 68.4	$+20.4 \pm 2.4$

(c) − (a) $= -3.7$, interval $[-5.2, -2.2]$; (c) − (b) $= -2.6 \pm 1.6$; (c) − (b) rounded $= +20.9 \pm 2.2$
4-bit start − bf16 start $= -6.5 \pm 2.8$
start rounded with GPTQ instead (no RL run): 51.2; GPTQ start − bf16 start $= -3.5 \pm 2.7$

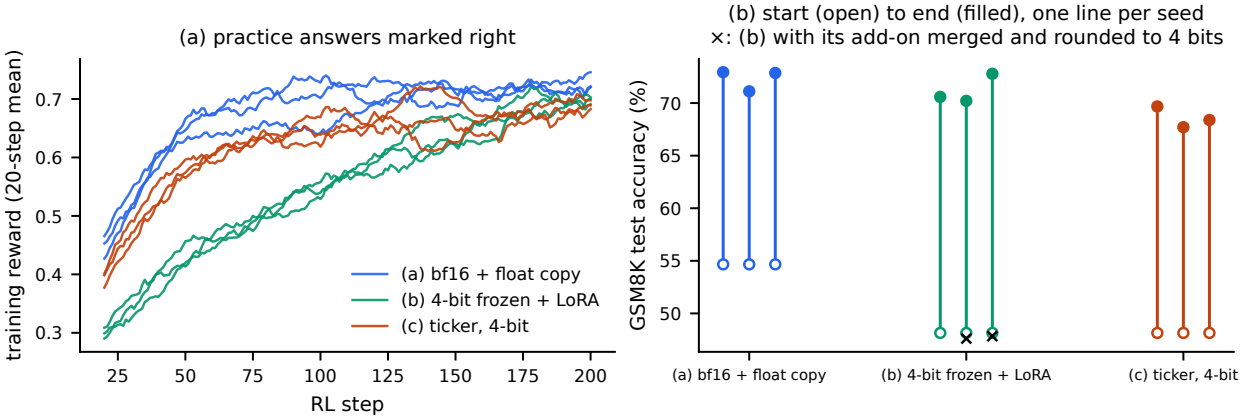


Figure 5: P5: (a) training reward, 20-step running mean, every run; (b) GSM8K test accuracy at the start and after 200 steps, one line per seed; ×: (b) with its adapter merged and rounded to 4 bits (seeds 2–3).

matrix stored in 4 bits with an 8-bit carry, trained with Muon. Setups (b) and (c) start from the same model rounded to nearest. The registered Muon learning rate was 10^{-4}, with a fallback rule fixed in advance: if the training reward of (a) seed 1, averaged over 20 steps, ended more than 0.04 below its peak, every run would use $4 \cdot 10^{-5}$ instead. At 10^{-4} the reward of (a) peaked near step 35 and then collapsed, and its test accuracy fell from 54.7% to 36.6%, so all runs reported here use $4 \cdot 10^{-5}$. The rule used training reward only. A third seed of (a) and (c) was added by a rule fixed before any run.

Results. The ticker ends 3.7 points behind full-precision training, so the lower end of the interval, −5.2, misses the registered threshold of −4 (Table 9). Most of the gap is present before training, since rounding the model to 4 bits costs 6.5 points. That cost depends on how the start is rounded. Rounded with GPTQ instead, calibrated on 128 GSM8K training solutions, the starting model scores 51.2%, 3.0 ± 2.8 points above round to nearest and 3.5 ± 2.7 below bf16; we measured this after the RL runs and did not run RL from it. The ticker then gains 20.4 points against 17.6 for (a), although a model recovering from rounding has an easier start than one improving from a higher score, so this does not show that it learns faster. Every weight learns while stored in 4 bits, with 0.11% of the 4-bit values changing per step.

Generator and trainer compute with the same numbers. Both convert each 4-bit value times its

Table 10: P6: memory of one full-parameter training step of Qwen3.5-0.8B on a T4 (GiB).

setup	peak	weights + state	peak / standard
bf16 + AdamW with fp32 copy (standard)	11.74	9.83	1.00
float Muon	10.33	6.56	0.88
bf16-SR, 8-bit momentum	4.31	2.40	0.37
bf16-SR, updated layer by layer	3.96	2.40	0.34
ticker	**3.74**	**2.18**	**0.318**

fp32 block scale to bf16 in the same way; with FP8 block scales, as in the released optimizer, that product is exact in bf16. The remaining difference between their token probabilities (mean 0.0033 for the ticker, 0.0030 for (a)) comes from the two programs' arithmetic, and is as small as in the full-precision setup.

LoRA learns more slowly at first (Appendix D) but ends 2.6 points ahead of the ticker after 200 steps (interval $[-4.2, -1.0]$ for ticker minus LoRA). The two are scored differently: (b) keeps its adapter in 16 bits, as QLoRA-style models are served, while the ticker is scored with 4-bit weights only. To see how much of LoRA's gain survives in a pure 4-bit model, at the end of (b)'s seeds 2 and 3 we merged the adapter into the 4-bit weights and rounded to the same grid, recomputing the block scales. We registered the prediction that most weights would round back to their start and the model would score near its 48.1% start. In fact the adapter averages 0.5% of the gap between neighbouring 4-bit values, *no* 4-bit value changes, and the rounded model scores 47.7%, 23.8 ± 2.3 points below the same model with its adapter and 20.9 ± 2.2 below the ticker. This measures where the learning is stored, since QLoRA-style models are served with their adapter. LoRA's gain is real but held in 16 bits, while the ticker's is held in the 4-bit weights. Peak GPU memory in these runs is set by the generator's reserved share and the activations (27–30 GB for (a) and (c), 24 GB for (b), which needs no gradients or optimizer state for its frozen weights). The 4-bit values are held in fp32 tensors here, so P6 rather than P5 measures the method's memory.

5.6 P6: training memory

Setup. We measure one full-parameter training step of Qwen3.5-0.8B (752M parameters, 498M of them in the decoder matrices) on a T4 with a batch of 2×1024 tokens, using the released optimizer with packed storage: two 4-bit codes per byte, FP8 block scales, an int8 carry and int8 momentum. The update is applied to each matrix during the backward pass and its gradient freed at once, and activations are checkpointed.

Results. The ticker peaks at 0.318 of the standard setup (Table 10). Against bf16-SR the saving is small: 0.87 of its peak, 0.95 against bf16-SR with the same layer-by-layer update, and 0.91 in steady state. The decoder matrices alone shrink by $2.59/3.03 = 0.86$, but the embedding is a third of this model and is stored the same way in both. In plain, unfused PyTorch the update adds 4.8 s to a 13.5 s step. A full-parameter step of Qwen3.5-2B also fits on the T4, at 8.34 GiB, where the standard setup runs out of memory.

6 Discussion

When the ticker helps. The clearest result is negative for error feedback through the momentum. With Muon, an injected error is erased in part, redistributed, or allowed to overwrite the

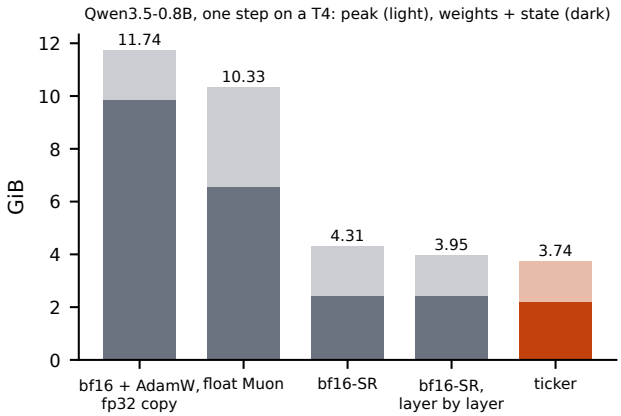


Figure 6: P6: peak memory (light) and weights plus optimizer state (dark) of one training step.

gradient, and ECO-Muon freezes the weights. Keeping the error per weight avoids the optimizer altogether, and an 8-bit carry is enough to match a float copy in every setting we tried. Against training in higher precision and rounding afterwards, the advantage depends on how much the final rounding loses. In continued training of a pretrained model, rounding after training loses 0.04 with round-to-nearest and still about 0.01 with GPTQ, and the ticker avoids that loss by training on the grid. From scratch at 45M parameters, GPTQ recovers almost the whole rounding loss, and bf16-SR followed by GPTQ matches the ticker for about 9% more training state. We do not know which property of the two settings decides this; model size, training length and the starting point all differ between P1 and P2.

What native 4-bit weights buy. The ticker reduces peak memory by only 5–13% against bf16-SR. Its main practical difference is that the trained weights are the 4-bit weights. In RL this means that the generator and the trainer use the same values without a correction for mismatch, and that what the model learns is stored in the 4-bit weights rather than in a 16-bit adapter that would vanish if merged and rounded. The cost is the starting point. Rounding Qwen3.5-0.8B to nearest costs 6.5 points, and 200 steps recover about two fifths of that relative to full-precision training. A better initial rounding, such as GPTQ, or a short phase of ordinary training before RL, might close the remaining gap; neither was tested.

Carry width. The registered noise model was wrong in a useful way. On our reading of the flip rates, the damage of a narrow carry comes from weights that sit next to a rounding boundary after a tick and are flipped back and forth by the carry's own random rounding, a mechanism the model leaves out. This suggests that the required width is set by the ratio between the carry unit and the step size near the boundary, and that schemes which keep weights away from the boundary after a tick could use fewer bits.

7 Related work

Training without master weights. ECO [20] removes the master copy for SGD with momentum and for Adam by injecting the rounding error into the momentum. It proves exact equivalence to master weights for SGD with momentum under an exact but memory-costly rule and gives a noise-floor bound for the memory-free rule. We show why the same injection fails for Muon. Error

Diffusion [1], which appeared days before this work, replaces the latent weight of a binary network with a 2–8-bit residual buffer updated by stochastic rounding, flips a weight when the buffer crosses a threshold, and reports that the method works best when the buffer grid has a level exactly at that threshold; it trains binary language models of up to about 19M binary parameters. The ticker is the multi-level form of that buffer: the threshold becomes half the gap to the neighbouring 4-bit value on each side, and the buffer is measured in units of that gap. What is new here is the analysis of Muon and of why error feedback through its momentum fails, 4-bit E2M1 grids with block scales, experiments at GPT-2 scale, the study of carry width, and full-parameter RL. Earlier, Bop [9] argued that binary networks need no latent weights at all and trained them with a momentum-threshold rule.

Compensated summation and stochastic rounding. Kahan summation [14] carries the low-order part lost in each addition. Zamirai et al. [31] show that pure bf16 training matches fp32 if weight updates use stochastic rounding or Kahan summation, and Collage [30] represents weights as several low-precision components to avoid fp32 copies. Stochastic rounding for deep learning goes back at least to Gupta et al. [8]; Croci et al. [3] survey its theory and uses. Our bf16-SR and naive 4-bit baselines are of this kind. The ticker differs by keeping the compensation term per weight, in few bits, and in units of the main term's own local resolution.

Error feedback in compressed optimization. Carrying a compression error forward to later steps is the core of 1-bit SGD [25] and of error-feedback methods for sign and compressed gradients [15]. There the error lives in the gradient communicated between machines; here it lives in the weights.

Muon and low-bit Muon. Muon [13] orthogonalises the momentum of each matrix with a Newton–Schulz iteration, and Liu et al. [18] scale it to large language models. MuonQ [27] and 4-bit Muon with GRASP [28] shrink Muon's *optimizer state* to 4 bits, both noting that orthogonalisation amplifies quantization error in the momentum (GRASP locates it in the top singular subspace); they keep full-precision weights. Our analysis is the weight-side counterpart: an error deliberately fed into the momentum does not come out intact.

4-bit training and post-training quantization. NVFP4 [21] and the microscaling formats [24] define the 4-bit values and block scales we use. Full-Stack FP4 [5] pretrains with 4-bit projections, optimizer states and attention but keeps bf16 master weights. FlashOptim [7] reduces optimizer memory and splits the master weight into lower-precision parts with a bound on the resulting error, so a bf16-level master weight still exists. GPTQ [6] rounds a trained model to low precision column by column, compensating each column's error with second-order information; we use it as the strongest form of rounding after training.

RL with quantized weights. QLoRA [4] fine-tunes a frozen 4-bit model through a 16-bit low-rank adapter [11], and QeRL [12] applies this to RL with an NVFP4 model; our setup (b) follows it. The trained model is then the 4-bit base plus a 16-bit adapter: QeRL's own evaluation attaches the adapter to the NVFP4 model in vLLM, or merges it into a 16-bit model. Merging the adapter into the 4-bit weights and rounding again can erase the adaptation, because the nearest grid point is usually the base itself. Scale-QLoRA [17] reports losses of up to 39 points from this and avoids them by training only the block scales, freezing every 4-bit code. The ticker goes the other way: every 4-bit code may change, and the trained model is the 4-bit model, with no adapter to merge.

In our LoRA setup the loss from merging and rounding is total, with no 4-bit value changing and the whole 23.8-point gain lost (Section 5.5). FlashRL [19] generates rollouts with a quantized copy and corrects the mismatch with the trainer by truncated importance sampling [29]. With the ticker, trainer and generator use the same bf16 values of the 4-bit weights, so there is no rounding mismatch between them, though other arithmetic differences remain.

8 Limitations

- **Scale.** The largest training runs are GPT-2 small (124M parameters, continued training) and a 45M model from scratch; RL uses Qwen3.5-0.8B. Behaviour at billions of parameters is not tested.
- **Memory.** Against bf16 weights with stochastic rounding and 8-bit momentum, the ticker saves only 5–13% at the peak. Against a frozen 4-bit model with LoRA it uses more training memory, since every weight needs a gradient and a carry.
- **RL.** One model, one task, 200 steps, three seeds. Whether the gap to full precision keeps closing with longer training, with a better initial rounding (a GPTQ start begins 3.0 points higher, but RL from it was not run), or with a short phase of ordinary training before RL is not tested. LoRA's learning rate was not tuned. The generator holds a bf16 copy of the 4-bit values.
- **GPTQ.** One configuration (natural column order, 1% damping, 128 calibration windows, no search over scales), applied to one seed of each baseline and to P5's starting model. A stronger post-training method may do better; from scratch the plain version already matches the ticker.
- **Simulation.** Training runs keep the 4-bit values in fp32 tensors and do not pack codes; only P6 measures packed storage. Block scales are fp32 during training; FP8 scales are counted but not trained with (a small character-level test found them to cost little; Appendix B). The update is not fused into a kernel.
- **Mechanism.** The measurement uses one model, an optimizer state built over a short window, and one error direction (the rounding remainder of the pretrained weights) plus a random control; it does not follow the error through many steps of training.
- **Carry width.** The registered noise model failed both tests, and the explanation offered for the flip rates was formed after seeing the data.
- **Only weights** are 4-bit. Activations and gradients stay in 16 or 32 bits, and embeddings, norms and biases are not quantized.

9 Conclusion

Error feedback through the momentum removes the full-precision copy for SGD and Adam, but not for Muon, whose orthogonalised update reshapes or overwrites an injected error. Keeping the error beside each weight, as an 8-bit carry in units of the weight's own grid spacing, trains 4-bit weights without a copy at under three bytes per weight and matches a float copy within about a hundredth of a nat, in continued training and from scratch; in continued training it is well ahead of ECO. Whether it also beats training in higher precision and rounding afterwards depends on the setting:

it does in continued training, even against GPTQ, but not from scratch at 45M parameters. In RL the ticker trains every weight of a 0.8B model in 4 bits and keeps what it learns in those weights, but the model it starts from has already lost accuracy to rounding. The open questions are scale, the starting point for RL, and how few carry bits a design that avoids boundary jitter could use.

Code and data. The optimizer with packed 4-bit storage, every experiment script, the registered plan with its dated changes, and the log of every run in this paper are at `https://github.com/NibblesOn/masterless`.

Broader impact. This work makes training and fine-tuning in low precision cheaper, which lowers the cost of adapting models for good and for harmful uses alike. It introduces no new data or models with specific risks.

Use of AI tools. The experiment code, the analysis scripts and this text were written with substantial help from an AI coding assistant (Claude, by Anthropic), working under the author's direction, and a draft was reviewed with another AI model (Gemini, by Google). The author chose the questions and the registered criteria, checked the results, and is responsible for all content.

References

[1] Pierre-Olivier Bonin. Error diffusion replaces latent weights: Training binary-weight language models on a few bits of dithered state. *arXiv preprint arXiv:2610.34566*, 2026.

[2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[3] Matteo Croci, Massimiliano Fasi, Nicholas J. Higham, Theo Mary, and Mantas Mikaitis. Stochastic rounding: Implementation, error analysis and applications. *Royal Society Open Science*, 9(3):211631, 2022. doi: 10.1098/rsos.211631.

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

[5] Siyu Ding, Mingchuan Ma, Jiabo Tong, Xingrun Xing, Ziming Wang, and Guoqi Li. Full-stack FP4: Stable LLM pretraining with quantized projections, optimizers, and attention. *arXiv preprint arXiv:2607.04422*, 2026.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.17323.

[7] Jose Javier Gonzalez Ortiz, Abhay Gupta, Christopher Rinard, and Davis Blalock. FlashOptim: Optimizers for memory-efficient training. *arXiv preprint arXiv:2602.23349*, 2026.

[8] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep learning with limited numerical precision. *arXiv preprint arXiv:1502.02551*, 2015.

[9] Koen Helwegen, James Widdicombe, Lukas Geiger, Zechun Liu, Kwang-Ting Cheng, and Roeland Nusselder. Latent weights do not exist: Rethinking binarized neural network optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[10] Nicholas J. Higham. *Functions of Matrices: Theory and Computation*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2008. doi: 10.1137/1.9780898717778.

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[12] Wei Huang, Yi Ge, Shuai Yang, Yicheng Xiao, Huizi Mao, Yujun Lin, Hanrong Ye, Sifei Liu, Ka Chun Cheung, Hongxu Yin, Yao Lu, Xiaojuan Qi, Song Han, and Yukang Chen. QeRL: Beyond efficiency – quantization-enhanced reinforcement learning for LLMs. *arXiv preprint arXiv:2510.11696*, 2025.

[13] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024.

[14] William Kahan. Further remarks on reducing truncation errors. *Communications of the ACM*, 8(1):40, 1965. doi: 10.1145/363707.363723.

[15] Sai Praneeth Karimireddy, Quentin Rebjock, Sebastian U. Stich, and Martin Jaggi. Error feedback fixes SignSGD and other gradient compression schemes. In *International Conference on Machine Learning (ICML)*, 2019.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[17] Tung-Ling Li, Jiale Huang, Lee-Chi Wang, and Janaki Ram Gotei. Scale-QLoRA: Code-invariant adapter merging for native 4-bit microscaling LLMs. *arXiv preprint arXiv:2609.04526*, 2026.

[18] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for LLM training. *arXiv preprint arXiv:2502.16982*, 2025.

[19] Liyuan Liu, Feng Yao, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. FlashRL. https://fengyao.notion.site/flash-rl, August 2025.

[20] Mahdi Nikdan, Amir Zandieh, Dan Alistarh, and Vahab Mirrokni. ECO: Quantized training without full-precision master weights. *arXiv preprint arXiv:2601.22101*, 2026.

[21] NVIDIA, Felix Abecassis, Anjulie Agrusa, Dong Ahn, et al. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025.

[22] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

[23] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[24] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[25] Frank Seide, Hao Fu, Jasha Droppo, Gang Li, and Dong Yu. 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs. In *Proc. Interspeech*, pages 1058–1062, 2014. doi: 10.21437/Interspeech.2014-274.

[26] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[27] Yupeng Su, Ruijie Zhang, Ziyue Liu, Yequan Zhao, and Zheng Zhang. MuonQ: Enhancing low-bit muon quantization via directional fidelity optimization. *arXiv preprint arXiv:2605.11396*, 2026.

[28] Huaijin Wu, Bingrui Li, Yebin Yang, Yi Tu, Zhanpeng Zhou, Jianfei Chen, and Junchi Yan. Achieving low-bit Muon through subspace preservation and grid quantization. In *International Conference on Learning Representations (ICLR)*, 2026. Code: https://github.com/wuhuaijin/lowbit-Muon.

[29] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your efficient RL framework secretly brings you off-policy RL training. https://fengyao.notion.site/off-policy-rl, August 2025.

[30] Tao Yu, Gaurav Gupta, Karthick Gopalswamy, Amith Mamidala, Hao Zhou, Jeffrey Huynh, Youngsuk Park, Ron Diamant, Anoop Deoras, and Luke Huan. Collage: Light-weight low-precision strategy for LLM training. In *International Conference on Machine Learning (ICML)*, 2024.

[31] Pedram Zamirai, Jian Zhang, Christopher R. Aberger, and Christopher De Sa. Revisiting BFloat16 training. *arXiv preprint arXiv:2010.06192*, 2020.

A Proof of Proposition 1

Write $U = HO$ with $H = (UU^\top)^{1/2} = P\Sigma P^\top$ symmetric positive definite and $O = \mathrm{polar}(U) = PQ^\top$, so $OO^\top = I_m$. Both factors are smooth functions of U near a full-rank point. Differentiating $U = HO$ and $OO^\top = I$ gives

$$\delta = dH\, O + H\, dO, \qquad dO\, O^\top + O\, dO^\top = 0.$$

Let $\Omega = P^\top dO\, Q$ and $S = P^\top dH\, P$, which is symmetric. Since $O^\top P = Q$, the second equation gives $P^\top dO\, O^\top P + P^\top O\, dO^\top P = \Omega + \Omega^\top = 0$, so Ω is antisymmetric. Multiplying the first equation by $P^\top$ on the left and Q on the right, and using $OQ = P$,

$$\tilde{\delta} = S + \Sigma\, \Omega.$$

Its transpose is $\tilde{\delta}^\top = S - \Omega\Sigma$, and subtracting the two gives $\tilde{\delta} - \tilde{\delta}^\top = \Sigma\Omega + \Omega\Sigma$, that is $\Omega_{ij}(\sigma_i + \sigma_j) = \tilde{\delta}_{ij} - \tilde{\delta}_{ji}$, so $\Omega = K$. The symmetric part of $\tilde{\delta}$ enters only S. For the part of dO outside the row space of U, note $O(I - QQ^\top) = 0$, so multiplying the first equation by $(I - QQ^\top)$ on the right gives $\delta(I - QQ^\top) = H\, dO\, (I - QQ^\top)$, that is $dO\, (I - QQ^\top) = P\Sigma^{-1}P^\top \delta (I - QQ^\top)$. Finally $dO = dO\, QQ^\top + dO\, (I - QQ^\top) = PKQ^\top + P\Sigma^{-1}P^\top \delta (I - QQ^\top)$, which is Equation (6). □

B Earlier small-scale results

These runs led to the method. They use a character-level GPT on Shakespeare (4 layers, width 128, 0.8M parameters, 2,000 steps; loss in nats per character, mean ± standard error over 3 seeds) and are not evidence for any main claim. The numbers are in the repository under `runs/m1`–`m4`, `runs/m7`, `runs/ticker` and `runs/twobit`.

Muon and ECO. Float Muon reached 1.6278±0.0032 and AdamW 1.9150±0.0034. 4-bit weights with a float copy and Muon reached 1.6463 ± 0.0019, and without a copy, with stochastic rounding, 1.7636 ± 0.0044. ECO with Adam reached 1.9283 ± 0.0039, close to its own float AdamW, as the ECO paper reports. ECO's injection adapted to Muon, with the error added to the momentum before Newton–Schulz, reached 1.8981 ± 0.0057, *worse* than no feedback at all. Measured during training, the change Newton–Schulz made in response to the injected error had a cosine of about 0.69 with the error (0.691, 0.692 and 0.692 for the 3 seeds), and the error grew from 0.15 to 0.50 of the momentum's size as the learning rate decayed. Adding the error after Newton–Schulz instead, from a 2-byte buffer, reached 1.6733 ± 0.0043.

The ticker. On a T4 in fp16, a 4-bit carry (2.28 bytes per weight) reached 1.6501 ± 0.0045, against 1.6458 ± 0.0035 for 4-bit with a float copy (8 bytes), and beat both a 4-bit buffer with stochastic rounding of the weight (1.7120 ± 0.0019) and FP8 training rounded to 4 bits afterwards (1.6717 ± 0.0043) in every seed.

2-bit weights. With 2-bit weights an 8-bit carry reached 1.7476 ± 0.0059 against 1.7253 ± 0.0015 for a float copy with Muon, and a 4-bit carry reached 1.8027 ± 0.0040.

A first GPT-2 run with a 4-bit carry. Continued training of GPT-2 small for 100M tokens with a 4-bit carry (seed 1, T4, fp16) ended $+0.0465 \pm 0.0010$ above 4-bit with a float copy, outside a 0.03 margin. Between 1.6% and 1.9% of the 4-bit values changed at every step regardless of the learning rate, the same flip rate later measured in P3 (Table 7). In a probe with random gradients, an 8-bit carry removed the effect, which is why the main experiments use an 8-bit carry.

C Changes to the registered plan

The plan, its criteria and every change are dated in `PAPER_PLAN.md` in the repository. No criterion was changed after a result it judges was seen. Table 11 lists every change that affects what was run or how it is judged.

D The 30-step RL design

For a time the plan judged P5 on a shorter design: 30 steps, 64 questions per step, 8 answers each, Muon at 10^{-4}. It was completed after the switch back to 200 steps and is reported here in full, but it is not used for any criterion. Setup (a) seed 1 of this design was the best of three learning rates on the test set, which favours (a).

Table 11: Changes to the plan, in order. "Before" means before any run that the change affects.

date	change	when, and why
10-28, rev. 2–3	P5a: "mean of 2 seeds at least (a) -2 points" became "lower end of the 95% interval of (c) $-$ (a) above -4"; P5 re-scoped; P6 reduced to 1.5B; bf16-SR baseline added	before any run; outside feedback
10-28, rev. 3	RL model Qwen2.5-0.5B-Instruct replaced by Qwen3.5-0.8B, and P6's model by Qwen3.5-0.8B on a T4; rule for P5's third seeds added; 2-bit runs and 124M from scratch dropped	before any run; the author's decision and compute
10-28	three details made precise by the code (Kahan's bytes 5.78 rather than 4.78; the correctness checks)	before any P0–P6 run
10-28	RL carry: 8 bits rather than the 12 bits the noise model predicted (P3c), so P3c is not tested	after P3, before P5; the noise model had failed both tests
10-29	questions per RL step reduced from 64 to 32 by the plan's own pilot re-costing rule; optimizer Muon (plan's rule); learning rate 10^{-4} with a fallback rule set in advance	after the pilot, before P5
10-29	P5 switched to a 30-step design to save compute, then back to the 200-step design of rev. 3; the 30-step design was completed and is reported (Appendix D)	one 30-step result of (c) was seen before the switch back
10-29	the fallback rule triggered at 10^{-4}, so every 200-step run uses $4 \cdot 10^{-5}$	the rule used training reward only
10-29	third seeds of (a) and (c), by the rule; LoRA seeds 2–3, and LoRA merged and rounded to 4 bits (measured, with a registered prediction)	after seeds 1–2, before these runs
10-29	ECO with AdamW added to P1 with a new criterion, P1e	after an outside review, before these runs
10-29	GPTQ applied to the pretrained start, to float Muon and bf16-SR of P1 and to bf16-SR of P2 (seed 1 each), with a prediction and a rule: if any GPTQ model is not above the ticker by twice the paired s.e., the claim narrows to rounding to nearest	after P1e, before these runs; the author's decision
10-30	P5's starting model rounded with GPTQ and scored, without RL, with a prediction (about half of the 6.5 points recovered)	after P5; the author's decision

E Compute and reproducibility

Hardware. P1–P5 and the GPTQ runs used one NVIDIA A100-SXM4-40GB on Google Colab, with several runs sharing the GPU at once; the mechanism measurement (Section 3.3) and P6 used one Tesla T4; the small-scale studies of Appendix B used a 6-core CPU and a T4. Software: PyTorch 2.11 and transformers 5.20.2; RL generation used vLLM.

Run times (wall clock while sharing the GPU). P1: about 13 minutes per run (median of 20). P2: about 3 hours per run. P3: about 1 minute per run; P4: about 3 minutes. P5: 50–65 minutes per 200-step run. GPTQ: about 1 minute per model after retraining, 4 minutes for Qwen3.5-0.8B. The mechanism measurement took under a minute on the T4.

Total. The paper's experiments used about 176 Colab compute units, against 90 planned. This includes runs that are not reported in the main tables: the ECO-Muon run that overflowed in fp16, the RL run at learning rate 10^{-4} that triggered the fallback rule, the RL pilot, the 30-step design,

Table 12: The 30-step design: GSM8K test accuracy (%), Qwen3.5-0.8B. Intervals as in Table 9.

setup	start	end (per seed)
(a) bf16 + fp32 copy	54.7	70.3, 71.9
(b) 4-bit frozen + LoRA	48.1	52.4
(c) ticker	48.1	62.5, 66.6
(c) − (a) = −6.60, interval [−8.53, −4.66]		
(c) gain = +16.38, interval [+13.74, +19.01]; (a) gain = $+16.45 \pm 2.48$		
(c) − (b) = $+12.13 \pm 2.54$		

the retraining of three models for GPTQ, and about 10 units lost to a session that stayed idle after its runs had finished.

Reproducing. Every run is deterministic given its seed on the same GPU type and software versions: retraining the float Muon and bf16-SR models of P1 and P2 for GPTQ reproduced the logged losses exactly. The repository lists the command for every run.

F Licences

GPT-2 small (MIT licence), FineWeb (ODC-By 1.0), GSM8K (MIT), Qwen3.5-0.8B (Apache 2.0) and vLLM (Apache 2.0) were used under their licences, in their versions on the Hugging Face Hub and PyPI in October 2026.